

Sprawozdanie z projektu

System księgarni internetowej - Bookstore API

Backend REST oparty o Hono.js, Drizzle ORM i PostgreSQL

Autory: Bartłomiej Suchowian, Andrei Fedaruk

Data: 16.06.2026

Spis treści

1.	Wprowadzenie i cel projektu	3
1.1.	Zakres funkcjonalny	3
2.	Architektura aplikacji	4
2.1.	Przepływ żądania	4
2.2.	Walidacja wejścia	4
3.	Model danych	5
3.1.	Relacje	5
4.	Endpointy API	6
4.1.	Przykład utworzenia książki	6
4.2.	Przykład aktualizacji zamówienia	6
5.	Logika transakcji sprzedaży	7
5.1.	Scalanie pozycji	7
5.2.	Sprawdzenie książek i magazynu	7
5.3.	Obliczenie wartości zamówienia	7
6.	Raportowanie i widoki bazodanowe	8
6.1.	Raport przychodu według kategorii	8
6.2.	Ranking książek	8
6.3.	Ranking użytkowników	8
6.4.	Uwagi do raportów	8
7.	Uruchomienie i praca z projektem	9
7.1.	Przykładowe testy ręczne	9
7.2.	Testy automatyczne	9
7.3.	Zidentyfikowane ograniczenia	9
8.	Propozycje rozwoju	11
8.1.	Paginacja i filtrowanie	11
9.	Podsumowanie	11
10.	Bibliografia i źródła projektu	12

1. Wprowadzenie i cel projektu

Repozytorium projektu znajduje się na stronie <https://github.com/afedaruk/bookstore/>.

Projekt bookstore jest aplikacją backendową przeznaczoną do obsługi podstawowych procesów księgarni internetowej. System nie zawiera pełnego frontendu, ale udostępnia komplet endpointów HTTP, które mogą zostać wykorzystane przez aplikację kliencką, panel administracyjny albo narzędzia testowe typu Postman lub curl.

Głównym celem implementacji było zbudowanie czytelnego API dla operacji CRUD oraz dodanie logiki biznesowej związanej ze sprzedażą książek. W systemie występują trzy grupy funkcji. Pierwsza grupa to zarządzanie słownikami i encjami podstawowymi, czyli książkami, kategoriami oraz użytkownikami. Druga grupa to obsługa zamówień i pozycji zamówień. Trzecia grupa to raportowanie, obejmujące między innymi zestawienie książek według kategorii, przychód według kategorii, najczęściej kupowane książki oraz użytkowników, którzy wydali najwięcej pieniędzy.

Aplikacja została napisana w TypeScript. Do obsługi żądań HTTP użyto frameworka Hono, który zapewnia prosty mechanizm routingu i dobrze pasuje do małych usług REST. Dostęp do bazy danych realizowany jest przez Drizzle ORM. Baza danych działa w PostgreSQL uruchamianym przez Docker Compose. Walidacja danych wejściowych została wykonana przy pomocy biblioteki Zod oraz adaptera @hono/zod-validator.

1.1. Zakres funkcjonalny

Zakres implementacji obejmuje następujące obszary:

- obsługę książek: dodawanie, pobieranie, aktualizacja i usuwanie,
- obsługę kategorii: pobieranie listy oraz tworzenie nowych kategorii,
- obsługę użytkowników: dodawanie, pobieranie, aktualizacja i usuwanie,
- obsługę zamówień: tworzenie, pobieranie, aktualizacja statusu oraz usuwanie,
- obsługę pozycji zamówienia: pełne operacje CRUD,
- tworzenie transakcji sprzedaży w jednej transakcji bazodanowej,
- raportowanie sprzedaży przez widoki bazodanowe,
- dokumentację API w formacie OpenAPI i interfejs Swagger UI.

2. Architektura aplikacji

Aplikacja ma architekturę warstwową. Warstwa wejściowa składa się z endpointów HTTP tworzonych przez Hono. Każdy moduł API jest osobną podaplikacją, która następnie zostaje podpięta w `src/index.ts` pod konkretną ścieżką bazową. Dzięki temu kod związany z książkami nie miesza się z kodem dotyczącym użytkowników, zamówień lub raportów.

```
app.route("/books", booksApp);
app.route("/categories", categoriesApp);
app.route("/reports", reportsApp);
app.route("/users", usersApp);
app.route("/orders", ordersApp);
app.route("/order-items", orderItemsApp);
app.route("/transactions", transactionsApp);
```

Warstwa serwisowa zawiera funkcje wykonujące zapytania Drizzle ORM. Dla prostych zasobów są to standardowe operacje `select`, `insert`, `update` oraz `delete`. Dla zamówień zastosowano transakcję przy usuwaniu, ponieważ razem z zamówieniem należy usunąć jego pozycje.

2.1. Przepływ żądania

Typowy przepływ żądania wygląda następująco:

```
Klient HTTP
|
v
Endpoint Hono w src/api
|
v
Walidacja Zod
|
v
Funkcja serwisowa w src/service
|
v
Drizzle ORM
|
v
PostgreSQL
```

2.2. Walidacja wejścia

Walidacja została wykonana blisko warstwy HTTP. Przykładowo identyfikatory w ścieżce są konwertowane do liczb całkowitych dodatnich:

```
const paramSchema = z.object({
  id: z.coerce.number().int().positive(),
});
```

W przypadku kwot pieniężnych zastosowano tekst walidowany wyrażeniem regularnym. Jest to spójne z faktem, że Drizzle dla kolumn typu `decimal` często zwraca wartości jako string, aby uniknąć utraty precyzji w JavaScript. Przykładowy format ceny dopuszcza do ośmiu cyfr przed przecinkiem i maksymalnie dwie cyfry po kropce.

```
price: z
  .string()
  .max(11)
  .regex(/^\d{1,8}(\.\d{1,2})?$/)
```

3. Model danych

Model danych jest relacyjny. Główne encje to kategorie, książki, użytkownicy, zamówienia oraz pozycje zamówień. Schemat jest zdefiniowany w `src/schema.ts` z wykorzystaniem Drizzle ORM. W bazie występuje także typ wyliczeniowy `order_status`, który ogranicza możliwe statusy zamówienia.

Tabela	Najważniejsze pola	Rola
categories	id, name	Słownik kategorii książek. Pole name jest wymagane i unikalne.
books	id, title, price, stock, category_id	Katalog książek wraz z ceną, stanem magazynowym i opcjonalnym przypisaniem do kategorii.
users	id, name, email	Klienci systemu. Adres e-mail jest unikalny.
orders	id, user_id, status, created_at, total	Nagłówek zamówienia, powiązany z użytkownikiem i posiadający status oraz wartość całkowitą.
order_items	id, order_id, book_id, quantity, price_per_item	Pozycje zamówienia przechowujące książkę, ilość i cenę jednostkową w momencie sprzedaży.

3.1. Relacje

Relacje w aktualnym schemacie są następujące:

```
categories 1 --- n books
users      1 --- n orders
orders     1 --- n order_items
books      1 --- n order_items
```

4. Endpointy API

API jest podzielone na zasoby. Każdy moduł eksportuje aplikację Hono oraz fragment dokumentacji OpenAPI. W `src/index.ts` wszystkie fragmenty dokumentacji są łączone w jedną specyfikację dostępną pod adresem `/doc`. Interfejs Swagger UI jest dostępny pod `/ui`.

Ścieżka	Metody	Opis
<code>/books</code>	GET, POST	Pobranie listy książek oraz utworzenie nowej książki.
<code>/books/{id}</code>	GET, PATCH, DELETE	Pobranie, aktualizacja lub usunięcie pojedynczej książki.
<code>/categories</code>	GET, POST	Pobranie kategorii oraz utworzenie nowej kategorii.
<code>/users</code>	GET, POST	Pobranie użytkowników oraz dodanie użytkownika.
<code>/users/{id}</code>	GET, PATCH, DELETE	Operacje na pojedynczym użytkowniku.
<code>/orders</code>	GET, POST	Pobranie zamówień oraz ręczne utworzenie zamówienia.
<code>/orders/{id}</code>	GET, PATCH, DELETE	Pobranie, aktualizacja albo usunięcie zamówienia.
<code>/order-items</code>	GET, POST	Pobranie lub utworzenie pozycji zamówienia.
<code>/order-items/{id}</code>	GET, PATCH, DELETE	Operacje na pojedynczej pozycji zamówienia.
<code>/transactions</code>	POST	Utworzenie zamówienia razem z pozycjami i aktualizacją magazynu.
<code>/transactions/{id}</code>	GET	Pobranie transakcji według identyfikatora zamówienia.

4.1. Przykład utworzenia książki

POST `/books`

```
{
  "title": "Czysty kod",
  "price": "59.99",
  "stock": 12,
  "categoryId": 4
}
```

Jeżeli rekord zostanie poprawnie zapisany, API zwraca status 201 Created i utworzoną książkę. Jeżeli `categoryId` wskazuje nieistniejącą kategorię, baza zgłasza naruszenie klucza obcego, a globalny handler błędów zwraca komunikat o braku powiązanego zasobu.

4.2. Przykład aktualizacji zamówienia

PATCH `/orders/{id}`

```
{
  "status": "cancelled"
}
```

5. Logika transakcji sprzedaży

Moduł transactions pozwala utworzyć zamówienie i jego pozycje na podstawie listy książek oraz ilości. W przeciwieństwie do ręcznego tworzenia zamówienia i pozycji, endpoint transakcyjny wykonuje komplet operacji jako jedną całość.

```
const createTransactionSchema = z.object({
  userId: z.number().int().positive().max(2147483647),
  items: z.array(
    z.object({
      bookId: z.number().int().positive().max(2147483647),
      quantity: z.number().int().positive().max(2147483647),
    }),
  ).min(1),
});
```

Po utworzeniu zamówienia i pozycji system zmniejsza stany magazynowe.

5.1. Scalanie pozycji

Pierwszym etapem logiki jest scalenie powtarzających się pozycji. Jeżeli klient przekaże tę samą książkę kilka razy, system zsumuje ilości. Dzięki temu dalsze sprawdzanie magazynu i zapis pozycji działają na oczyszczonej liście.

```
const mergedItemsMap = new Map<number, number>();

for (const item of data.items) {
  const currentQuantity = mergedItemsMap.get(item.bookId) ?? 0;
  mergedItemsMap.set(item.bookId, currentQuantity + item.quantity);
}
```

To proste rozwiązanie zapobiega sytuacji, w której dwie pozycje tej samej książki osobno przechodzą walidację, ale łącznie przekraczają dostępny stan magazynowy.

5.2. Sprawdzenie książek i magazynu

Po scaleniu system pobiera książki z bazy przez inArray. Następnie sprawdza, czy liczba znalezionych książek jest równa liczbie żądanych identyfikatorów. Jeśli nie, rzuca błąd BOOK_NOT_FOUND. Kolejny etap sprawdza dostępność magazynową. Jeśli stan książki jest mniejszy niż żądana ilość, funkcja zgłasza błąd NOT_ENOUGH_STOCK z tytułem książki.

5.3. Obliczenie wartości zamówienia

Wartość zamówienia jest liczona jako suma iloczynów ceny i ilości. Wynik jest zapisywany w kolumnie total jako tekst sformatowany do dwóch miejsc po przecinku:

```
total += Number(book.price) * item.quantity;

.values({
  userId: data.userId,
  status: "pending",
  total: total.toFixed(2),
})
```

W prostym projekcie jest to wystarczające. W systemie produkcyjnym warto byłoby rozważyć wykonywanie obliczeń pieniężnych po stronie bazy albo operowanie na groszach jako liczbach całkowitych, aby uniknąć problemów precyzji typu number w JavaScript.

6. Raportowanie i widoki bazodanowe

Raporty zostały zaimplementowane jako widoki Drizzle w `src/schema.ts`. Takie podejście oddziela zapytania analityczne od zwykłych operacji CRUD i pozwala pobierać gotowe zestawienia przez proste funkcje serwisowe.

Widok	Znaczenie
<code>book_categories_view</code>	Lista książek wraz z kategorią, ceną i stanem magazynowym. Widok używa <code>rightJoin</code> , aby zachować książki nawet wtedy, gdy nie mają kategorii.
<code>category_revenue_view</code>	Zestawienie liczby sprzedanych pozycji i przychodu dla każdej kategorii. Wartości puste są zastępowane zerem przez <code>COALESCE</code> .
<code>top_books_view</code>	Ranking dziesięciu książek według sprzedanej ilości i przychodu. Pomija zamówienia anulowane.
<code>top_users_view</code>	Ranking dziesięciu użytkowników według łącznej wydanej kwoty i liczby zamówień. Pomija zamówienia anulowane.

6.1. Raport przychodu według kategorii

Widok `category_revenue_view` łączy kategorie, książki i pozycje zamówień. Dla każdej kategorii wylicza liczbę sprzedanych egzemplarzy oraz łączny przychód. W funkcji serwisowej wynik jest dodatkowo sortowany malejąco po przychodzie i ograniczany do 20 rekordów.

```
export async function getCategoryRevenue() {
  return await db
    .select()
    .from(categoryRevenueView)
    .orderBy(desc(categoryRevenueView.totalRevenue))
    .limit(20);
}
```

6.2. Ranking książek

Widok `top_books_view` grupuje dane po książce i sumuje ilość oraz przychód. Zastosowano `leftJoin`, dzięki czemu książki bez sprzedaży mogą nadal pojawiać się w zestawieniu z wartościami zerowymi. Warunek ignoruje anulowane zamówienia:

```
.where(sql`${orders.status} IS NULL OR ${orders.status} != 'cancelled'`)
```

Jest to istotne, ponieważ anulowane zamówienie nie powinno zwiększać wyniku sprzedażowego książki. Analogiczny warunek znajduje się w widoku `top_users_view`.

6.3. Ranking użytkowników

Widok `top_users_view` wylicza liczbę zamówień oraz sumę wartości zamówień dla każdego użytkownika. Wynik jest sortowany malejąco według `totalSpent` i ograniczony do dziesięciu rekordów. Endpoint `/reports/top-users` zwraca użytkowników, którzy wydali najwięcej pieniędzy.

6.4. Uwagi do raportów

Raporty są dobrym przykładem wykorzystania bazy danych do agregacji. Nie ma potrzeby pobierania wszystkich pozycji zamówień do aplikacji i sumowania ich w TypeScript. PostgreSQL wykonuje grupowanie i sumowanie efektywniej oraz bliżej danych. Ograniczeniem aktualnej wersji jest brak filtrowania po czasie, na przykład raportu dziennego, miesięcznego albo rocznego. Taki filtr można dodać przez kolumnę `created_at` w tabeli `orders`.

7. Uruchomienie i praca z projektem

Typowy scenariusz uruchomienia projektu lokalnie składa się z czterech kroków. Najpierw należy uruchomić bazę danych, następnie zainstalować zależności, wypchnąć lub wykonać migracje schematu, załadować dane startowe i uruchomić serwer.

```
docker compose up -d
bun install
bun run db:push
bun run db:seed
bun run dev
```

Po uruchomieniu serwer nasłuchuje lokalnie pod `http://localhost:3000/`.

7.1. Przykładowe testy ręczne

Poprawność API można sprawdzić ręcznie przez `curl`. Przykładowe żądanie pobrania książek:

```
curl http://localhost:3000/books
```

Przykładowe utworzenie transakcji:

```
curl -X POST http://localhost:3000/transactions \
-H "Content-Type: application/json" \
-d '{
  "userId": 1,
  "items": [
    { "bookId": 1, "quantity": 2 },
    { "bookId": 2, "quantity": 1 }
  ]
}'
```

Po wykonaniu takiej operacji można pobrać szczegóły transakcji przez:

```
curl http://localhost:3000/transactions/1
```

Następnie warto sprawdzić, czy stany magazynowe książek zmniejszyły się zgodnie z zamówionymi ilościami. Można też wywołać `/reports/top-books`, `/reports/category-revenue` oraz `/reports/top-users`, aby sprawdzić, czy raporty uwzględniają nową sprzedaż.

7.2. Testy automatyczne

W projekcie przygotowano testy jednostkowo-integracyjne dla serwisu transakcji z wykorzystaniem modułu `bun:test`. Przed każdym testem baza danych jest czyszczona z danych testowych, dzięki czemu każdy przypadek uruchamia się w izolowanym stanie i nie zależy od wyników poprzednich testów.

Testy sprawdzają najważniejsze scenariusze działania mechanizmu zamówień. Pierwszy przypadek weryfikuje poprawne utworzenie transakcji, obliczenie łącznej ceny oraz zmniejszenie stanu magazynowego książki. Drugi test sprawdza obsługę błędów w sytuacji, gdy użytkownik próbuje zamówić większą liczbę egzemplarzy niż dostępna w magazynie.

Dodatkowo testowane jest pobieranie transakcji po identyfikatorze zamówienia wraz z powiązanymi pozycjami zamówienia. Ostatni test sprawdza odporność systemu na sytuację wyścigu, w której dwóch użytkowników próbuje jednocześnie kupić ostatni dostępny egzemplarz książki. Oczekiwanym rezultatem jest powodzenie tylko jednej transakcji oraz odrzucenie drugiej, co potwierdza poprawność zabezpieczenia przed sprzedażą większej liczby książek niż dostępna w magazynie.

7.3. Zidentyfikowane ograniczenia

Najważniejsze ograniczenia aktualnej wersji to brak uwierzytelniania i autoryzacji. Każdy klient mający dostęp do API może tworzyć, edytować i usuwać zasoby. W rzeczywistym systemie należałoby rozdzielić uprawnienia klienta, administratora i pracownika sklepu. Szczególnie wrażliwe są operacje usuwania książek, zmiany zamówień i przeglądania użytkowników.

Drugim ograniczeniem jest brak automatycznego przywracania stanów magazynowych przy anulowaniu zamówienia. Endpoint `PATCH /orders/{id}` pozwala zmienić status na `cancelled`, ale nie wykonuje dodatkowej operacji magazynowej. Dla pełnej spójności biznesowej warto dodać dedykowany endpoint, na przykład `POST /orders/{id}/cancel`, który działałby transakcyjnie.

Trzecim ograniczeniem jest brak paginacji i filtrowania w większości list. Endpointy `GET /books`, `GET /users` i `GET /orders` zwracają wszystkie rekordy. Przy małych danych demonstracyjnych nie stanowi to problemu, ale przy większej bazie mogłoby obciążać aplikację i sieć.

8. Propozycje rozwoju

Projekt można rozwinąć w kilku kierunkach. Pierwszym kierunkiem jest poprawa procesów zamówieniowych. Warto dodać dedykowane operacje biznesowe: opłacenie zamówienia, wysłanie zamówienia, dostarczenie zamówienia i anulowanie zamówienia. Każda z nich powinna kontrolować poprawność przejścia statusu. Na przykład zamówienia `delivered` nie powinno dać się cofnąć do `pending` bez specjalnych uprawnień.

Drugim kierunkiem jest rozszerzenie modelu książki. Aktualnie książka ma tytuł, cenę, stan magazynowy i kategorię. W praktycznym systemie przydatne byłyby pola takie jak autor, ISBN, opis, wydawnictwo, rok wydania, okładka oraz data dodania.

Trzecim kierunkiem jest bezpieczeństwo. Należy dodać uwierzytelnianie użytkowników, hashowanie haseł, tokeny sesyjne lub JWT oraz kontrolę dostępu do endpointów administracyjnych. Operacje tworzenia książek, aktualizacji magazynu i raportowania nie powinny być publicznie dostępne.

8.1. Paginacja i filtrowanie

Dla endpointów listujących warto dodać parametry `limit`, `offset`, `page`, `sort` oraz filtry. Przykładowo `GET /books?categoryId=4&limit=20` mogłoby zwracać tylko książki z kategorii IT i programowanie. Raporty mogłyby otrzymać zakres dat, na przykład `from` i `to`, aby analizować sprzedaż w konkretnym okresie.

9. Podsumowanie

Projekt bookstore realizuje główne wymagania prostego backendu księgarni internetowej. Udostępnia endpointy REST dla podstawowych zasobów, wykorzystuje relacyjną bazę danych, waliduje dane wejściowe i posiada mechanizm raportowania. Najważniejszą funkcjonalnością jest endpoint transakcyjny, który tworzy zamówienie wraz z pozycjami, oblicza łączną wartość oraz aktualizuje magazyn w jednej transakcji bazodanowej.

10. Bibliografia i źródła projektu

Sprawozdanie zostało przygotowane na podstawie dostarczonego kodu źródłowego projektu bookstore.

- `src/index.ts` - konfiguracja aplikacji Hono, podpięcie tras i dokumentacji,
- `src/schema.ts` - definicje tabel, enuma statusu zamówienia oraz widoków raportowych,
- `src/api/*.ts` - endpointy HTTP, walidacja Zod i fragmenty OpenAPI,
- `src/service/*.ts` - operacje bazodanowe i logika transakcji,
- `src/utils/error.ts` - centralna obsługa błędów,
- `src/seed.ts` oraz `data/*.csv` - dane startowe,
- `docker-compose.yaml` - konfiguracja lokalnego PostgreSQL,
- `package.json` - zależności i skrypty uruchomieniowe.